

MicroC Compilador

Analizador Léxico desarrollado por **Carlos Maldonado**
Curso: Autómatas y Lenguajes · 2026 · Semestre V
Universidad Mesoamericana de Guatemala · Catedrático: Ing. Baudilio Boteo

1. Descripción del proyecto

MicroC Compilador implementa la **primera fase de un compilador**: el **analizador léxico**. Su objetivo es leer código fuente escrito en una variante de C/C++ y generar una lista de tokens estructurada, identificando qué elementos pertenecen al lenguaje y cuáles no.

El proyecto cumple con dos fases:

- **FASE I:** Generar lista de tokens, eliminar espacios/tabs/saltos de línea, relacionar líneas con el análisis.
- **FASE II:** Identificar palabras reservadas, números (enteros y reales) y comentarios.

2. Estructura de clases (UML)

Siguiendo el diagrama UML del PDF, el compilador está organizado en tres clases:

Clase	Responsabilidad
frmEditor	Frame de visualización gráfica con botones del menú
Analizador Léxico	Propiedades y funciones del análisis léxico
Unidades Léxicas	Tabla de símbolos del lenguaje (diccionario de tokens)

3. Clase Unidades Léxicas

Esta clase es la **tabla de símbolos** del compilador. Funciona como un diccionario que traduce cada palabra o símbolo a un número (token) entre 1 y 100.

Diccionario de palabras (self.Palabra)

```
self.Palabra = {  
    "auto": 1, "break": 2, "case": 3, "char": 4,  
    "if": 16, "int": 17, "return": 20, "while": 32,  
    "include": 40, "stdio.h": 42,  
    "printf": 50, "scanf": 51
```

```
}
```

Guarda **todas las palabras reservadas de C**, las librerías estándar (stdio.h, math.h, etc.) y las funciones más comunes (printf, scanf). Cada una tiene un número único.

Diccionario de símbolos (self.símbolos)

```
self._simbolos = {
    "+": 60, "-": 61, "*": 62,      # Aritméticos
    "=": 65, "==" : 66, "!=": 71,  # Relacionales
    "&&": 72, "||": 73,            # Lógicos
    "(" : 75, ")" : 76, "{" : 77,   # Agrupación
    ";": 92, ":", 93                # Misceláneos
}
```

Métodos

```
def GetTokenPalabra(self, lexema: str) -> int:
    """Si la palabra está en el diccionario, devuelve su token.
       Si NO está, devuelve 98 → es un IDENTIFICADOR del usuario."""
    return self.Palabra.get(lexema, 98)

def GetTokenSimbolo(self, lexema: str) -> int:
    """Si el símbolo está en el diccionario, devuelve su token.
       Si NO está, devuelve -1 → no pertenece al lenguaje."""
    return self._simbolos.get(lexema, -1)
```

4. Clase Analizador Léxico

Esta clase contiene **toda la lógica del análisis léxico**. Recorre el código fuente carácter por carácter y va clasificando lo que encuentra.

Atributos (Figura II del UML)

```
self.Lista = []          # Lista de tokens encontrados
self._cont = 0           # Contador de posición en el archivo
self._Linea = 1          # Número de línea actual
```

Funciones de alfabeto

```
def GetAlfabetoAlfanumerico(self, c):
    # ¿Es una letra o un guión bajo?
    return 1 if (c.isalpha() or c == '_') else 0
```

```

def GetAlfabetoNumero(self, c):
    # ¿Es un dígito?
    return 1 if c.isdigit() else 0

def GetAlfabetoSimbolo(self, c):
    # ¿Es un símbolo del lenguaje?
    return 1 if c in '+-*/=<>!&|^~(){}[];.,.' else 0

```

Este es el corazón del compilador. *Un conjunto de if que valida el símbolo y que está encerrado en un while que existe mientras haya caracteres en el archivo.*

```

def AnalisisLexico(self, archivo):
    self.Lista = []
    errores = []
    cont = 0
    linea = 1

    while cont < len(archivo):          # ← MIENTRAS HAYA CARACTERES
        c = archivo[cont]

        if c == '\n':                    # ¿Salto de línea?
            linea += 1; cont += 1; continue

        if c in ' \t\r':                  # ¿Espacio o tab?
            cont += 1; continue

        if self.GetAlfabetoAlfanumerico(c): # ¿Letra o _ ?
            cont = self.IdentificadorPalabraReservada(...)

        elif self.GetAlfabetoNumero(c):    # ¿Dígito?
            cont = self.EnteroReal(...)

        elif c == '/':                    # ¿Diagonal?
            cont, linea = self.AutomataComentario(...)

        elif self.GetAlfabetoSimbolo(c):    # ¿Símbolo?
            token = UL.GetTokenSimbolo(lexema)

        else:                              # ¡Desconocido!
            errores.append((linea, c))      # ← ERROR LÉXICO

```

6. Los 3 autómatas implementados Identificador

Palabra Reservada

Lee caracteres mientras sean letras, dígitos o guión bajo. Al terminar, consulta el diccionario para saber si es una palabra reservada (token 1-51) o un identificador del usuario (token 98).

```
def IdentificadorPalabraReservada(self, archivo, cont, linea, UL):
    lexema = ""
    while cont < len(archivo) and (archivo[cont].isalnum() or
archivo[cont] == '_'):
        lexema += archivo[cont]
        cont += 1
    token = UL.GetTokenPalabra(lexema)
    tipo = "IDENTIFICADOR" if token == 98 else "RESERVADA"
    self.Lista.append((linea, lexema, token, tipo))
    return cont
```

7. Clase frm Editor — Interfaz

Construye toda la ventana del compilador con tkinter. Sus funciones principales coinciden con el UML:

Función	Acción
OpcNuevo_Click	Crea un archivo nuevo en blanco
OpcAbrir_Click	Abre un .c desde el sistema de archivos
OpcGuardar_Click	Guarda el archivo actual
OpcGuardarComo_Click	Guarda con un nombre nuevo
OpcSalir_Click	Cierra la app (pregunta si hay cambios sin guardar)
compilarToolStripMenuItem_Click	Ejecuta el análisis léxico (Fase I + II)

8. Tabla completa de tokens (máx. 100)

Rango	Categoría	Ejemplos
1 – 33	Palabras reservadas	auto, if, int, return, while, main
40 – 41	Directivas preprocesador	include, define
42 – 46	Librerías estándar	stdio.h, math.h, string.h
50 – 51	Funciones integradas	printf, scanf
60 – 64	Op. aritméticos	+ - * / %
65 – 71	Asignación / relacionales	= == < > <= >= !=
72 – 74	Op. lógicos	&& !
75 – 80	Agrupación	() { } []
92 – 97	Misceláneos / delimitadores	;, . : # ' "

Rango	Categoría	Ejemplos
98	Identificador	variables del usuario
99	Número	42, 3.14
100	Cadena / carácter literal	"hola", 'a'

9. Ejemplo de ejecución

Si en el editor escribes:

```
// Programa de ejemplo
int main() {
    int edad = 25;
    printf("Hola");
    return 0;
}
```

Al presionar **F5**, el analizador genera esta tabla:

Línea	Lexema	Token	Tipo
2	int	17	PALABRA RESERVADA
2	main	33	PALABRA RESERVADA
2	(	75	OP. AGRUPACIÓN
2	)	76	OP. AGRUPACIÓN
2	{	77	OP. AGRUPACIÓN
3	edad	98	IDENTIFICADOR
3	=	65	OP. RELACIONAL O ASIGNACIÓN
3	25	99	NÚMERO
3	;	92	MISCELÁNEO
4	printf	50	FUNCIÓN INTEGRADA
4	"Hola"	100	CADENA

10. Bibliotecas utilizadas

Todas son parte de la **librería estándar de Python 3** — no se requiere instalar nada externo.

Biblioteca	Uso en el proyecto
tkinter	Crea la ventana, los menús, el editor de texto y los paneles
tkinter.ttk	Widgets modernos: Treeview (tabla de tokens), Notebook (pestañas)
tkinter.filedialog	Diálogos para abrir y guardar archivos .c
tkinter.messagebox	Mensajes emergentes (confirmaciones al salir)
re	Expresiones regulares para el resaltado de

Biblioteca	Uso en el proyecto
	sintaxis del editor